

ИНФОРМАТИКА

11 класс. Углублённый уровень

Глава

ТЕОРИЯ СЛОЖНОСТИ АЛГОРИТМОВ

Идеи, методы и границы возможного в мире вычислений

Содержание

- §1. Зачем нужна теория сложности. Идеи и история
- §2. Расширенный алгоритм Евклида и идея инварианта
- §3. Универсальное хеширование: рандомизация и средний случай
- §4. Сортировки и принцип «разделяй и властвуй»
- §5. Задача о рюкзаке: жадность, динамика, приближение
- §6. P, NP и за их пределами. Доказательства без разглашения
- Заключение
- Литература
- Задачи для самостоятельной работы

От авторов

Эта глава посвящена не столько отдельным алгоритмам, сколько способу мыслить о них. Мы редко задаём себе вопрос «можно ли в принципе решить эту задачу?» и почти всегда — «можно ли решить её достаточно быстро?». Между этими вопросами лежит граница, которую исследует наука, называемая теорией сложности вычислений.

Когда вы выбираете маршрут в навигаторе, поисковая система за доли секунды подбирает страницы по запросу, банк защищает ваш счёт от взлома, а врач получает результат расшифровки генома, в каждой из этих задач принципиально различается то, что мы вообще можем делать «за разумное время». Понимать, почему одни задачи решаются мгновенно, другие — за обозримое, но большое время, а третьи кажутся вне досягаемости даже для самых мощных суперкомпьютеров — значит понимать сам предмет информатики.

Мы будем стараться идти не от формулы к формуле, а от идеи к идее. Каждый раз, разбирая алгоритм, мы будем выделять метод, который за ним стоит: рекурсивное разделение, динамическое программирование, рандомизация, приближение, сводимость к другой задаче. Эти методы — настоящее богатство информатики, и владеть ими полезно далеко за пределами школьной программы.

В тексте нам встретится немало математики: логарифмы, рекуррентные соотношения, простые оценки в виде неравенств. Бояться этого не стоит — большая часть рассуждений требует лишь аккуратности и желания разобраться. Мы будем подкреплять каждую идею наглядными примерами, иллюстрациями и пошаговыми «прогонами» алгоритмов. В конце главы — задачи, которые позволят попробовать свои силы.

Что важно вынести из этой главы. Не количество выученных алгоритмов, а ощущение того, что у любого вычислительного процесса есть «цена», что эту цену можно измерять и сравнивать, и что грамотный выбор идеи иногда сокращает время работы программы в миллиарды раз.

§1. Зачем нужна теория сложности. Идеи и история

1.1. Алгоритм и его «цена»

Алгоритм — это конечная последовательность чётких инструкций, которая по входным данным шаг за шагом приводит к ответу. Мы пользовались алгоритмами с младших классов: складывали в столбик, делили уголком, искали наибольший общий делитель. Уже тогда вы могли заметить: один способ решения задачи требует «много операций», а другой — «мало». Например, чтобы перемножить два числа в столбик, мы выполняем количество элементарных умножений и сложений, пропорциональное произведению длин чисел. И это лучше, чем считать произведение через многократное прибавление, что было бы безнадежно медленно.

Чтобы говорить об эффективности строго, нам нужны две вещи: модель того, что считается «элементарной операцией», и способ измерять, сколько таких операций сделает алгоритм для входа размера n . Первое определяется выбранной моделью вычислений; второе — функцией $T(n)$, называемой временной сложностью алгоритма.

В качестве модели мы будем использовать привычную: процессор за единицу времени выполняет одну элементарную операцию (присваивание, сравнение, арифметическое действие с числами фиксированной длины, обращение к ячейке массива по индексу). Эта модель называется RAM-машиной (Random Access Machine) и хорошо отражает то, что делает реальный компьютер. Тонкости вроде кэшей, конвейеризации и параллелизма мы оставим в стороне: они меняют постоянные множители, но не суть.

1.2. Худший, средний, лучший случай

Время работы зависит не только от размера входа, но и от самого входа. Линейный поиск элемента в массиве из n чисел в лучшем случае находит ответ на первом шаге, а в худшем — обходит весь массив. Чтобы избежать произвола, обычно фиксируют один из трёх режимов оценки:

- **худший случай** (worst case): максимальное число операций по всем входам данного размера. Это «гарантия сверху», и именно она чаще всего интересует разработчиков критичных систем.
- **средний случай** (average case): среднее число операций, обычно при равномерном распределении входов. Чаще ближе к практике, но требует аккуратной формулировки модели случайности.
- **лучший случай** (best case): мало о чём говорит, но иногда полезен (например, чтобы оценить, насколько алгоритм «адаптивен» к удобным входам).

Особое место занимает амортизированный анализ, при котором редкие «дорогие» операции усредняются по последовательности «дешёвых». Знаменитый пример —

увеличение размера динамического массива (vector в C++, list в Python): каждое отдельное добавление в среднем стоит постоянное число операций, хотя изредка мы переписываем весь массив целиком.

1.3. Асимптотические обозначения: O , Ω , Θ

Точная функция $T(n)$ для конкретного алгоритма выглядит, скажем, так: $T(n) = 7n^2 + 30n + 12$. Однако на больших n решающую роль играет старший член, а постоянные множители зависят от того, насколько ловко мы реализуем алгоритм на конкретном языке и процессоре. Это привело к идее асимптотических обозначений (Бахман, конец XIX века; Эдмунд Ландау, начало XX; в информатику введены Дональдом Кнутом).

Пусть $f(n)$ и $g(n)$ — положительные функции натурального аргумента. Запись $f(n) = O(g(n))$ (читается « $f(n)$ есть O -большое от $g(n)$ ») означает, что найдутся такие постоянные $C > 0$ и n_0 , что для всех $n \geq n_0$ выполнено $f(n) \leq C \cdot g(n)$. Содержательно: при больших n функция f не превосходит g с точностью до постоянного множителя.

Симметричное обозначение $f(n) = \Omega(g(n))$ означает, что $f(n) \geq c \cdot g(n)$ при достаточно больших n и некоторой положительной константе c — оценка снизу. Совместное $f(n) = \Theta(g(n))$ означает «и сверху, и снизу того же порядка»: $c \cdot g(n) \leq f(n) \leq C \cdot g(n)$.

Например, для $T(n) = 7n^2 + 30n + 12$ верно $T(n) = O(n^2)$ и $T(n) = \Omega(n^2)$, значит $T(n) = \Theta(n^2)$. Зато про неё нельзя сказать $T(n) = O(n)$: подобрать константу C нельзя, ведь n^2 растёт быстрее n .

Удобство этих обозначений в том, что они отбрасывают «неинтересные» константы и младшие члены, оставляя главное — порядок роста.

1.4. Полиномиальная и экспоненциальная сложность

Алгоритм, время работы которого ограничено сверху некоторым многочленом от n , называется полиномиальным; класс таких алгоритмов — это P -сложность в широком смысле. Алгоритмы со сложностью 2^n , $n!$, n^n называют экспоненциальными. Граница между этими двумя мирами — главное противопоставление в теории сложности. Связано это с тем, что полиномиальное время удваивается при удвоении мощности компьютера, а экспоненциальное — добавляет к достижимому n всего лишь константу.

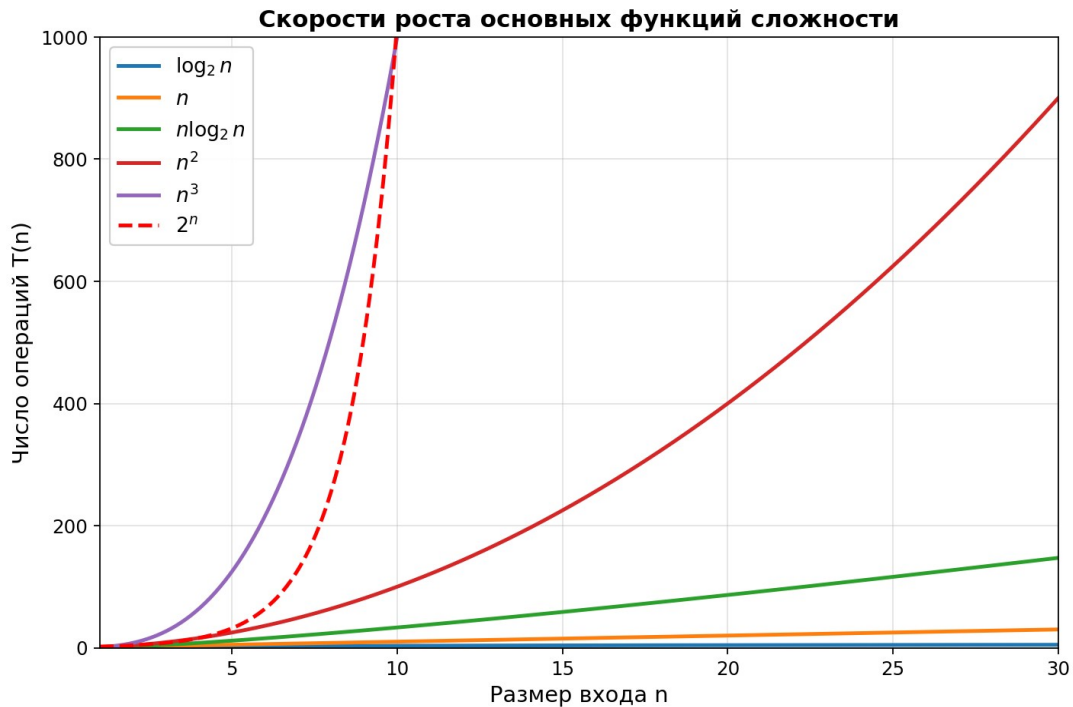


Рис. 1.1. Скорости роста основных функций сложности. Логарифм почти не виден внизу, экспонента быстро уходит за пределы графика.

Чтобы прочувствовать разницу, посмотрим, какое время потребовало бы выполнение указанного числа операций при типичной для современного процессора скорости — около миллиарда операций в секунду:

Время работы при 10^9 операций/с (полиномиальные vs экспоненциальные)

n	n	n^2	n^3	2^n	$n!$
10	10 нс	100 нс	1 мкс	1 мкс	3.6 мс
20	20 нс	400 нс	8 мкс	1 мс	77 лет
30	30 нс	900 нс	27 мкс	1 с	$8.4 \cdot 10^{15}$ лет
50	50 нс	2.5 мкс	125 мкс	13 дней	практически ∞
100	100 нс	10 мкс	1 мс	$4 \cdot 10^{13}$ лет	практически ∞
1000	1 мкс	1 мс	1 с	практически ∞	практически ∞

Рис. 1.2. Сравнение времени работы алгоритмов разной сложности. Полиномиальные оценки лежат в «зелёной» зоне, экспоненциальные при сколько-нибудь больших n превосходят возраст Вселенной.

Главный практический вывод: рост производительности компьютеров не отменяет необходимости думать об алгоритме. Наоборот, чем мощнее машины, тем больший выигрыш даёт замена экспоненциального алгоритма на полиномиальный.

1.5. Краткая история теории сложности

Корни теории алгоритмов уходят в логику и основания математики. В 1900 году Давид Гильберт сформулировал список открытых математических проблем; в десятой задаче он спрашивал, существует ли алгоритм, определяющий, имеет ли произвольное диофантово уравнение целочисленные решения. Позднее этот вопрос привёл к самому понятию алгоритма.

В 1936 году появились две независимые формализации: λ -исчисление Алонзо Чёрча и абстрактная вычислительная машина Алана Тьюринга. Тьюринг показал, что задача об остановке программы алгоритмически неразрешима — это был первый отрицательный результат о возможностях вычислений. Так возникла теория вычислимости, отвечающая на вопрос «что в принципе можно вычислить?».

После Второй мировой войны, когда появились первые ЭВМ, на первый план вышел другой вопрос — «что можно вычислить достаточно быстро?». В 1960-е годы Юрис Хартманис и Ричард Стирнс ввели меры временной и пространственной сложности и доказали иерархические теоремы: больше времени — действительно больше задач. Алан Кобэм и Джек Эдмондс предложили считать практически решаемыми именно задачи, для которых существует полиномиальный алгоритм.

В 1971 году Стивен Кук, а независимо в 1973 году Леонид Левин в СССР доказали ключевую теорему: задача о выполнимости булевой формулы (SAT) является «самой трудной» в классе NP — задач, ответ для которых легко проверить. Через год Ричард Карп показал, что 21 классическая задача — раскраска графа, гамильтонов цикл, задача коммивояжёра, целочисленное программирование — также «самая трудная». Возникла гипотеза $P \neq NP$, которая по сей день остаётся одной из главных нерешённых проблем математики и одной из задач тысячелетия Клэйвского математического института. За её решение объявлена премия в один миллион долларов.

Параллельно развивались и другие направления: рандомизированные алгоритмы (М. Рабин, Соловей и Штрассен), приближённые алгоритмы для NP-трудных задач (С. Сахни, А. Иббара), интерактивные доказательства (Ш. Голдвассер, С. Микали, Ч. Раккофф). Всё это сегодня — действующие, активно развивающиеся ветви науки.

1.6. Что мы будем делать дальше

В оставшихся параграфах мы разберём конкретные алгоритмы, каждый из которых иллюстрирует один или несколько фундаментальных приёмов:

1. **Алгоритм Евклида** — рекурсия и инвариант. Поразительно: древнейший алгоритм оказывается логарифмическим благодаря неочевидной связи с числами Фибоначчи.

2. **Универсальное хеширование** — случайность как средство получить хорошее ожидаемое время работы.
3. **Сортировки слиянием и быстрая** — принцип «разделяй и властвуй» и анализ через рекуррентные соотношения.
4. **Задача о рюкзаке** — жадность, динамическое программирование, приближённые алгоритмы.
5. **Классы P и NP** — понятие сводимости, NP-полнота, факторизация, простота, изоморфизм графов и доказательства без разглашения.

§2. Расширенный алгоритм Евклида и идея инварианта

2.1. Постановка задачи и наивная попытка

Наибольший общий делитель двух натуральных чисел a и b — это наибольшее натуральное число, на которое делятся оба. Обозначим его $\text{НОД}(a, b)$. Например, $\text{НОД}(12, 18) = 6$, потому что 6 делит и 12, и 18, а никакое большее число этим свойством не обладает.

Самый прямолинейный способ найти НОД — перебрать все возможные делители от 1 до $\min(a, b)$ и выбрать наибольший подходящий. На паре чисел вроде (12, 18) это работает мгновенно. Но как только числа становятся длиной 200 десятичных знаков (а именно такие используются в современной криптографии), этот подход требует выполнить около 10^{200} операций — больше, чем атомов во Вселенной. Нужен принципиально другой подход.

2.2. Алгоритм Евклида: идея и обоснование

Этот алгоритм описан в седьмой книге «Начал» Евклида около 300 года до нашей эры и, по-видимому, был известен ещё раньше. Идея настолько проста, что её можно выписать в одну строчку:

$$\text{НОД}(a, b) = \text{НОД}(b, a \bmod b), \text{ если } b > 0; \quad \text{НОД}(a, 0) = a.$$

Здесь $a \bmod b$ — остаток от деления a на b . Почему это равенство верно? Запишем $a = q \cdot b + r$, где $0 \leq r < b$. Если d делит и a , и b , то d делит и $r = a - q \cdot b$, то есть d — общий делитель пары (b, r) . И наоборот: если d делит b и r , то d делит $a = q \cdot b + r$. Значит, множества общих делителей у пар (a, b) и (b, r) совпадают, а тогда совпадают и их наибольшие элементы.

Иначе говоря, переход от (a, b) к $(b, a \bmod b)$ сохраняет НОД — это и есть инвариант алгоритма. На каждом шаге второй аргумент строго уменьшается (потому что $0 \leq r < b$), значит, рано или поздно мы получим пару $(a', 0)$ — и тогда ответ просто равен a' .

2.3. Геометрический смысл

У алгоритма Евклида есть прекрасная геометрическая интерпретация. Представим себе прямоугольник со сторонами a и b и попробуем замостить его одинаковыми квадратами наибольшего возможного размера, причём начнём с того, что отрезем от прямоугольника максимальное число квадратов со стороной b . После этого останется прямоугольник со сторонами b и $a \bmod b$. Повторим то же действие с ним, и так далее. Когда остаток исчезнет — это и будет сторона искомого квадрата, то есть НОД.

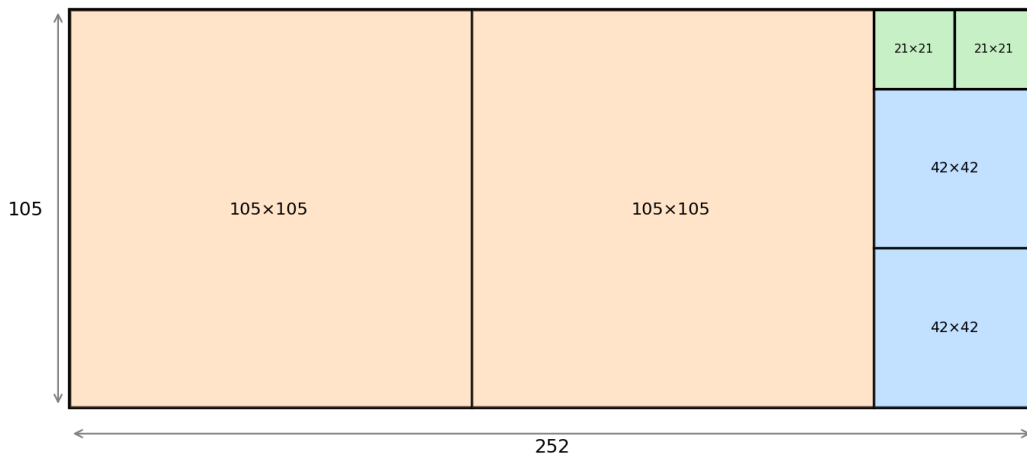
Геометрический смысл алгоритма Евклида: НОД(252, 105) = 21

Рис. 2.1. Алгоритм Евклида для $\text{НОД}(252, 105) = 21$. Прямоугольник 252×105 разрезается сначала на два квадрата 105×105 , остаётся 42×105 . Тот разрезается на два квадрата 42×42 , остаётся 42×21 . Тот делится ровно на два квадрата 21×21 без остатка.

2.4. Псевдокод

Запишем алгоритм в виде программы. Существует две естественные формы: рекурсивная и итеративная.

```
// Рекурсивная форма
function gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a mod b)
```

```
// Итеративная форма
function gcd_iter(a, b):
    while b != 0:
        a, b = b, a mod b
    return a
```

2.5. Конкретный пример

Найдём $\text{НОД}(252, 105)$ и проследим работу алгоритма по шагам:

```
gcd(252, 105): 252 = 2 · 105 + 42 → gcd(105, 42)
gcd(105, 42): 105 = 2 · 42 + 21 → gcd(42, 21)
gcd(42, 21): 42 = 2 · 21 + 0 → gcd(21, 0)
gcd(21, 0): возвращаем 21.
```

Ответ: $\text{НОД}(252, 105) = 21$. Заметьте, что мы выполнили всего четыре деления — куда лучше, чем обходить все 105 потенциальных делителей перебором.

2.6. Сложность: связь с числами Фибоначчи

Сколько шагов делает алгоритм Евклида в худшем случае? Этот вопрос решил в 1844 году французский математик Габриэль Ламе. Он заметил, что наихудшим входом являются именно последовательные числа Фибоначчи.

Напомним: $F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5, F_6 = 8, F_7 = 13, \dots$ (каждое следующее равно сумме двух предыдущих). Применим алгоритм к (F_{n+1}, F_n) :

$$\gcd(F_{n+1}, F_n) = \gcd(F_n, F_{n-1}) = \gcd(F_{n-1}, F_{n-2}) = \dots = \gcd(F_2, F_1) = \gcd(1, 0) = 1$$

То есть для входа (F_{n+1}, F_n) алгоритм делает ровно n шагов. С другой стороны, можно показать (это и есть теорема Ламе), что если алгоритм делает n шагов на каких-то входах $a > b$, то $a \geq F_{n+2}$.

Числа Фибоначчи растут экспоненциально: $F_n \approx \varphi^n / \sqrt{5}$, где $\varphi = (1 + \sqrt{5})/2 \approx 1.618$ — золотое сечение. Значит, число шагов n ограничено величиной порядка $\log_{\varphi} a$, то есть сложность алгоритма Евклида в худшем случае — $O(\log \min(a, b))$. Если меряться битовой длиной чисел $L = \log_2 a$, то это $O(L)$. Каждое вычисление остатка для чисел длиной L бит занимает $O(L^2)$ операций (школьное деление), так что общая битовая сложность — $O(L^3)$. Для чисел из 200 цифр (≈ 660 бит) это меньше миллиарда операций — доли секунды.

Теорема Ламе (1844). Число шагов алгоритма Евклида для пары (a, b) с $a \geq b > 0$ не превосходит количества цифр меньшего числа, умноженного на 5. Это первая в истории нетривиальная оценка сложности алгоритма.

2.7. Расширенный алгоритм: представление НОД через линейную комбинацию

Замечательное свойство НОД, открытое Этьеном Безу, состоит в следующем: для любых целых a и b существуют такие целые x и y , что

$$a \cdot x + b \cdot y = \text{НОД}(a, b).$$

Расширенный алгоритм Евклида не только находит сам НОД, но и явно вычисляет коэффициенты x и y . Это пригодится в самых разных задачах: от решения линейных диофантовых уравнений до построения обратных элементов в модулярной арифметике, без чего невозможны ни криптография с открытым ключом, ни современные вычисления в конечных полях.

Идея и вывод

Будем поддерживать инвариант: на каждом шаге мы храним пару (r_k, r_{k+1}) — текущие остатки, и две пары коэффициентов (x_k, y_k) и (x_{k+1}, y_{k+1}) , такие что

$$r_k = a \cdot x_k + b \cdot y_k,$$

$$r_{k+1} = a \cdot x_{k+1} + b \cdot y_{k+1}.$$

В начале $r_0 = a$, $r_1 = b$, поэтому естественно положить $(x_0, y_0) = (1, 0)$ и $(x_1, y_1) = (0, 1)$. На очередном шаге пишем $r_k = q_k \cdot r_{k+1} + r_{k+2}$ и получаем $r_{k+2} = r_k - q_k \cdot r_{k+1}$. Подставляя выражения через a и b , видим:

$$x_{k+2} = x_k - q_k \cdot x_{k+1},$$

$$y_{k+2} = y_k - q_k \cdot y_{k+1}.$$

Когда очередной остаток обнулится ($r_n = 0$), тогда r_{n-1} — это НОД, а его коэффициенты (x_{n-1}, y_{n-1}) — искомые.

Псевдокод

```
function ext_gcd(a, b):
    // возвращает (d, x, y), такие что d = НОД(a, b) и a·x + b·y = d
    x_prev, x_curr = 1, 0
    y_prev, y_curr = 0, 1
    r_prev, r_curr = a, b
    while r_curr != 0:
        q = r_prev div r_curr
        r_prev, r_curr = r_curr, r_prev - q · r_curr
        x_prev, x_curr = x_curr, x_prev - q · x_curr
        y_prev, y_curr = y_curr, y_prev - q · y_curr
    return (r_prev, x_prev, y_prev)
```

Пошаговый пример: $a = 252$, $b = 105$

Заполним таблицу значений переменных. На каждой строке записаны частное q и текущие (r, x, y) :

шаг	q	r	x	y	проверка $a \cdot x + b \cdot y$
0	—	252	1	0	$252 \cdot 1 + 105 \cdot 0 = 252$
1	—	105	0	1	$252 \cdot 0 + 105 \cdot 1 = 105$
2	2	42	1	-2	$252 \cdot 1 + 105 \cdot (-2) = 42$
3	2	21	-2	5	$252 \cdot (-2) + 105 \cdot 5 = 21$
4	2	0	5	-12	$252 \cdot 5 + 105 \cdot (-12) = 0$

Таблица 2.1. Пошаговая работа расширенного алгоритма Евклида для $(252, 105)$.

Когда r -столбец впервые становится нулём, ответ — это предыдущая строка: $\text{НОД}(252, 105) = 21$, причём $252 \cdot (-2) + 105 \cdot 5 = -504 + 525 = 21$. Действительно, $x = -2$, $y = 5$ — искомые коэффициенты Безу.

2.8. Зачем нужны коэффициенты Безу

Расширенный алгоритм Евклида — это «отмычка» к двум классическим задачам.

Решение линейного диофантова уравнения $a \cdot x + b \cdot y = c$

Уравнение имеет целочисленные решения тогда и только тогда, когда c делится на $d = \text{НОД}(a, b)$. В этом случае одно частное решение получается умножением коэффициентов Безу на c/d , а общее имеет вид

$$x = x_0 + (b/d) \cdot t, \quad y = y_0 - (a/d) \cdot t, \quad t - \text{любое целое.}$$

Обращение элемента по модулю

Пусть мы хотим найти такое y , что $a \cdot y \equiv 1 \pmod{m}$. Это равносильно решению уравнения $a \cdot y + m \cdot k = 1$, которое имеет решение тогда и только тогда, когда $\text{НОД}(a, m) = 1$ (то есть a и m взаимно просты). Расширенный алгоритм Евклида сразу даёт y . Эта операция — основа алгоритма RSA: чтобы построить пару «секретный — открытый ключ», участник криптосистемы выбирает большое число e и находит обратное $d = e^{-1} \pmod{\phi(N)}$, где $N = p \cdot q$, а ϕ — функция Эйлера.

Идея, ради которой стоило проделать этот разбор. В простой рекурсии $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ скрыт мощный инвариант — сохранение множества общих делителей. А лёгкая модификация (отслеживать ещё пару коэффициентов) даёт целое семейство применений в теории чисел и криптографии. Это типичный пример того, как продуманный анализ алгоритма открывает новые возможности.

§3. Универсальное хеширование: рандомизация и средний случай

3.1. Хеш-таблицы и проблема коллизий

Хеш-таблица — структура данных, которая позволяет хранить пары «ключ — значение» и быстро искать значение по ключу. Идея проста: каждому ключу x сопоставляется некоторое число $h(x) \in \{0, 1, \dots, m - 1\}$, называемое хешем. По этому числу определяется ячейка таблицы, в которой хранится значение, связанное с x . Если хеш-функция h хорошо «перемешивает» ключи, операции вставки, удаления и поиска занимают в среднем $O(1)$ — независимо от размера таблицы! Это намного быстрее, чем сбалансированные деревья поиска ($O(\log n)$).

Однако таблица имеет конечный размер m , а множество возможных ключей U обычно огромно (например, все возможные строки длины до 100). Значит, по принципу Дирихле, найдутся два ключа $x \neq x'$ с одинаковым хешем $h(x) = h(x')$. Такая ситуация называется коллизией. Один из распространённых способов борьбы с ними — метод цепочек: в каждой ячейке таблицы хранится список ключей, имеющих такой хеш.

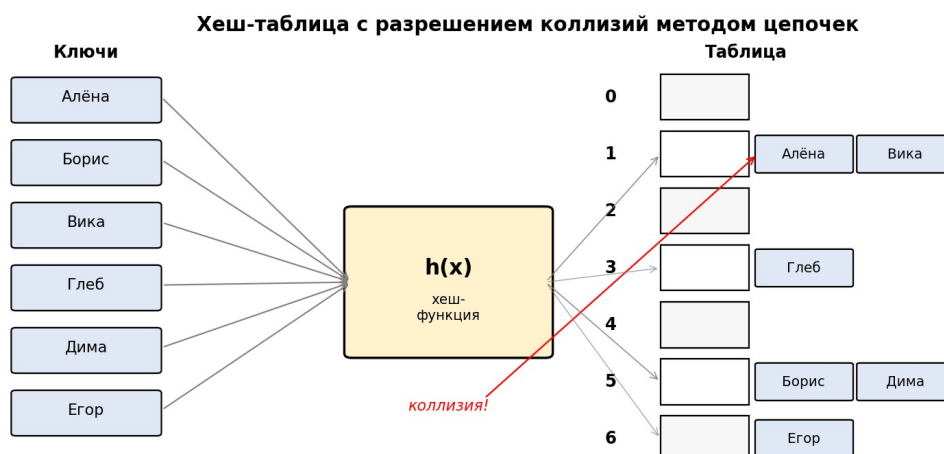


Рис. 3.1. Хеш-таблица из 7 ячеек. Ключи «Алёна» и «Вика» попали в одну ячейку 1, «Борис» и «Дима» — в ячейку 5. Их связанные списки и образуют цепочки.

Что произойдёт, если все наши ключи попадут в одну и ту же ячейку? Тогда мы фактически храним их в обыкновенном связанном списке, а поиск займёт линейное время $O(n)$ — как если бы хеш-таблицы и не было.

3.2. Почему «случайно выбранная» хеш-функция — это рискованно

Может показаться, что достаточно подобрать «хорошую» детерминированную хеш-функцию, которая равномерно перемешивает ключи. К сожалению, это иллюзия. Вот строгая претензия:

Утверждение. Какую бы детерминированную хеш-функцию $h: U \rightarrow \{0, \dots, m-1\}$ мы ни выбрали (даже самую хитрую!), всегда найдутся $n = |U|/m$ ключей в нашем универсуме, попадающих в одну ячейку. Если злоумышленник подаст на вход нашей программы именно эти ключи, поиск будет работать за $O(n)$.

Доказательство — простой подсчёт. Если m ячеек содержат всего $|U|$ ключей, то по крайней мере одна ячейка содержит не меньше $|U|/m$ из них. Это и есть «плохой» вход. Если хеш-функция фиксирована, противник всегда может его найти и атаковать систему.

В реальной жизни такие атаки случаются: в 2003 году исследователи показали, что многие популярные хеш-таблицы (в Perl, PHP) уязвимы к подобным атакам, и злонамеренные запросы могли «подвесить» сервер. Решение: не фиксировать хеш-функцию заранее, а выбирать её случайно из заранее заготовленного семейства. Если злоумышленник не знает, какая именно функция будет выбрана, он не сможет подготовить плохой вход.

3.3. Идея универсального семейства

Семейство хеш-функций $H = \{h_1, h_2, \dots, h_k\}$ называется универсальным, если для любых двух различных ключей $x \neq y$ вероятность коллизии случайно выбранной из H функции h не превосходит $1/m$:

$$\Pr_{\{h \in H\}} [h(x) = h(y)] \leq 1/m \text{ для всех } x \neq y.$$

Это очень естественное условие: «случайная» функция должна вести себя так, как если бы значения $h(x)$ и $h(y)$ выбирались независимо и равномерно из $\{0, \dots, m-1\}$ (тогда вероятность совпадения равна $1/m$). Универсальные семейства гарантируют такое поведение хотя бы для пар ключей.

3.4. Конструкция Картера—Вегмана

В 1979 году Лесли Картер и Марк Вегман предложили простое и красивое универсальное семейство. Зафиксируем простое число $p > |U|$ и натуральное число m (размер таблицы). Для произвольных $a \in \{1, 2, \dots, p-1\}$ и $b \in \{0, 1, \dots, p-1\}$ определим

$$h_{\{a,b\}}(x) = ((a \cdot x + b) \bmod p) \bmod m.$$

Семейство $H = \{ h_{\{a,b\}} : 1 \leq a < p, 0 \leq b < p \}$ содержит $p \cdot (p - 1)$ различных хеш-функций. Каждая из них выбирается одной и той же формулой, и достаточно случайно сгенерировать пару (a, b) , чтобы получить случайную h из H .

Доказательство универсальности

Пусть $x \neq y$ — два различных ключа из $\{0, \dots, p-1\}$. Подсчитаем число пар (a, b) , при которых $h_{\{a,b\}}(x) = h_{\{a,b\}}(y)$.

Обозначим $u = (a \cdot x + b) \bmod p$ и $v = (a \cdot y + b) \bmod p$. Из этих двух соотношений ($\bmod p$) получаем

$$u - v \equiv a \cdot (x - y) \pmod{p}, \quad \text{причём } x - y \not\equiv 0 \pmod{p}, \quad \text{так как } 0 < |x - y| < p.$$

Поскольку p простое, разность $x - y$ обратима по модулю p . Значит, для каждой пары (u, v) с $u \neq v$ ровно одно значение a удовлетворяет уравнению (a именно $a = (u - v) \cdot (x - y)^{-1} \bmod p$), и это однозначно определяет b . Значит, отображение $(a, b) \leftrightarrow (u, v)$ — биекция между $\{1, \dots, p-1\} \times \{0, \dots, p-1\}$ и $\{(u, v) : u \neq v\}$.

Теперь спросим: при скольких парах (u, v) с $u \neq v$ будет $u \bmod m = v \bmod m$? Для каждого фиксированного u таких v ровно $(p - 1)/m$ возможных значений (приблизительно — но не больше $\lceil p/m \rceil - 1$). Грубо: вероятность того, что случайно выбранные различные u и v одинаковы по модулю m , не превосходит $(p/m - 1)/(p - 1) \leq 1/m$.

Это и есть искомое неравенство $\Pr[\text{коллизия}] \leq 1/m$. Семейство универсально.

3.5. Конкретный пример

Возьмём $p = 17$, $m = 5$ и случайные $a = 3$, $b = 7$. Хешируем ключи 4, 11, 25, 30, 41:

ключ x	$3 \cdot x + 7$	$(3 \cdot x + 7) \bmod 17$	ячейка $((\cdot) \bmod 5)$	событие
4	19	2	2	новая ячейка
11	40	6	1	новая ячейка
25	82	14	4	новая ячейка
30	97	12	2	коллизия с 4!
41	130	11	1	коллизия с 11!

Таблица 3.1. Пример хеширования при $a=3$, $b=7$, $p=17$, $m=5$. Часть ключей попала в одну ячейку — это нормально: семейство гарантирует лишь ограничение по вероятности.

При другом выборе (a, b) распределение по ячейкам поменялось бы. Гарантия универсальности означает: для любых двух конкретных ключей доля «плохих» пар (a, b) — тех, что вызывают коллизию — не превосходит $1/m$.

3.6. Анализ ожидаемого времени поиска

Пусть в таблицу размера m вставлено n ключей и для каждого ключа хеш-функция выбрана случайно из универсального семейства. Оценим математическое ожидание длины цепочки.

Зафиксируем какой-нибудь ключ y и посчитаем, сколько других ключей попадёт с ним в одну ячейку. Введём индикаторные случайные величины: для каждого $x \neq y$ из набора положим $I_x = 1$, если $h(x) = h(y)$, и 0 иначе. По свойству универсальности $E[I_x] = \Pr[h(x) = h(y)] \leq 1/m$. По линейности математического ожидания число коллизий с ключом y равно

$$E[\sum_x I_x] = \sum_x E[I_x] \leq (n - 1) / m.$$

Если выбрать $m = \Theta(n)$ (то есть размер таблицы пропорционален числу ключей), то ожидаемая длина цепочки — константа, и поиск ключа в среднем работает за $O(1)$. Это и есть тот выигрыш, ради которого мы боролись.

Заметим: гарантия — в среднем по выбору хеш-функции, а не по входу. Это принципиально отличается от анализа в среднем по входам: какие бы ключи ни подал противник, мы всё равно получим $O(1)$ в среднем — потому что теперь случайность встроена в сам алгоритм.

Большая идея: рандомизация. Случайность как ингредиент алгоритма позволяет получать гарантии «в среднем» против любого, даже самого неудачно подобранного, входа. Это сильнее, чем средний случай по равномерному распределению. Идея рандомизированных алгоритмов очень часто используется в задачах, где детерминированные решения известны лишь медленные или сложные.

3.7. Где это применяется на практике

- **Хеш-таблицы стандартных библиотек.** Современные реализации (например, в Python, Go, Rust) используют хеш-функции с встроенной рандомизацией: при старте программы генерируется случайный «соляной» параметр, без знания которого подобрать атакующий вход практически невозможно.
- **Структуры с быстрыми частотными запросами.** Универсальное хеширование лежит в основе фильтров Блума и счётчиков Count-Min, которые экономят память при анализе больших потоков данных.
- **Распределённые вычисления.** Согласованное хеширование в системах вроде Cassandra или Memcached использует те же идеи для равномерного распределения ключей по серверам.

§4. Сортировки и принцип «разделяй и властвуй»

4.1. Зачем сортировать

Сортировка — операция упорядочивания элементов по некоторому ключу — одна из самых востребованных в программировании. После сортировки массив можно прямо за $O(\log n)$ обходить бинарным поиском, в нём легко найти медиану, обнаружить дубликаты, объединить с другим отсортированным массивом за линейное время. Поэтому неудивительно, что задача сортировки стала эталоном для проверки идей анализа алгоритмов.

Простейшие методы — пузырьковая сортировка, сортировка вставками, сортировка выбором — работают за $O(n^2)$ операций. Это годится для нескольких десятков элементов, но безнадёжно медленно для миллионов. Принципиально лучшие алгоритмы — сортировка слиянием и быстрая сортировка — основаны на одной общей идее, которую мы сейчас и обсудим.

4.2. Принцип «разделяй и властвуй»

Многие сложные задачи можно решить так:

6. **разделить** вход на несколько частей меньшего размера;
7. **решить** каждую подзадачу рекурсивно тем же методом;
8. **объединить** решения подзадач в решение исходной задачи.

Этот метод и называется «разделяй и властвуй» (divide and conquer). Если научиться сводить задачу размера n к двум задачам размера $n/2$ за линейное время, то она в целом решается за $O(n \log n)$. Это очень быстрый рост: при удвоении n время работы алгоритма не удваивается, а лишь чуть-чуть прибавляется.

4.3. Сортировка слиянием

Идея сортировки слиянием — реализация принципа «разделяй и властвуй» в чистом виде. Алгоритм был предложен Джоном фон Нейманом в 1945 году.

```
function merge_sort(A):
    if length(A) ≤ 1: return A
    mid = length(A) div 2
    L = merge_sort(A[0 ... mid-1])
    R = merge_sort(A[mid ... length(A)-1])
    return merge(L, R)

function merge(L, R):
    out = пустой массив длины |L| + |R|
    i = 0; j = 0; k = 0
    while i < |L| and j < |R|:
        if L[i] ≤ R[j]:
            out[k] = L[i]; i += 1
```

```

else:
    out[k] = R[j]; j += 1
    k += 1
while i < |L|: out[k] = L[i]; i += 1; k += 1
while j < |R|: out[k] = R[j]; j += 1; k += 1
return out

```

Идея процедуры merge: два уже отсортированных массива можно объединить в один отсортированный за линейное время, проходя по ним двумя указателями и каждый раз выбирая наименьший из доступных элементов.

Пример

Применим алгоритм к массиву [38, 27, 43, 3, 9, 82, 10]. Дерево рекурсии будет таким:

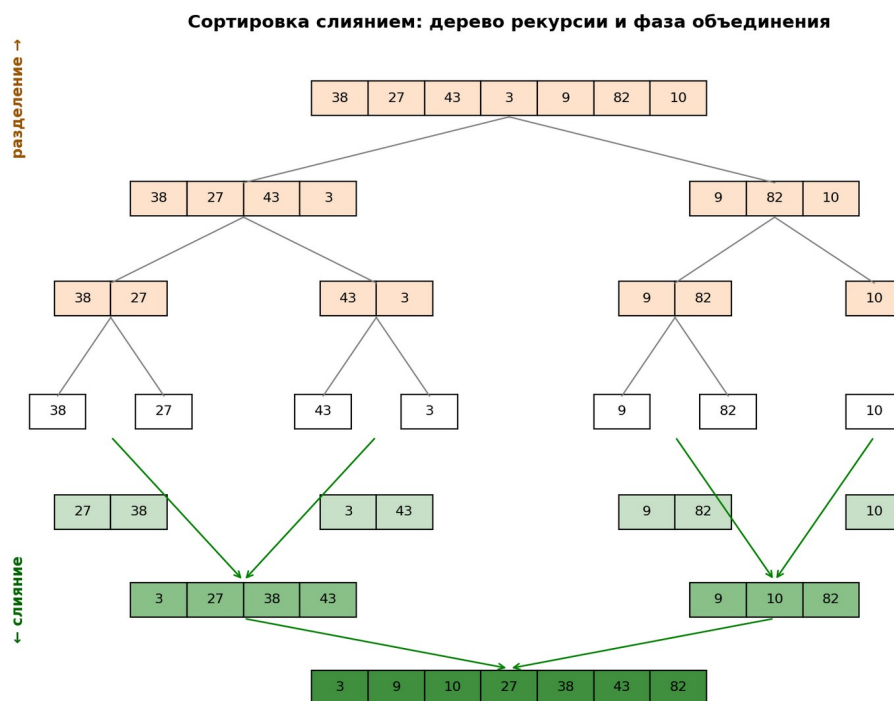


Рис. 4.1. Сортировка слиянием. На фазе разделения (оранжевые блоки) массив рекурсивно делится на части по одному элементу. На фазе слияния (зелёные блоки) пары упорядочиваются и объединяются — снизу вверх.

Анализ сложности

Обозначим $T(n)$ — время сортировки массива из n элементов. Алгоритм делает два рекурсивных вызова на массивах размера $n/2$ и одно слияние за $O(n)$ операций. Получаем рекуррентное соотношение

$$T(n) = 2 \cdot T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Решим его методом дерева рекурсии. На уровне 0 у нас один узел стоимости n . На уровне 1 — два узла по $n/2$, суммарно тоже n . На уровне 2 — четыре узла по $n/4$,

суммарно n . И так далее. Уровней всего $\log_2 n + 1$ (пока массив не сократится до одного элемента). На каждом уровне суммарная работа равна $\Theta(n)$. Итого $T(n) = \Theta(n \log n)$.

Эта оценка справедлива и для худшего, и для лучшего, и для среднего случая: алгоритм всегда выполняет «то же самое» дерево рекурсии независимо от входа. Сортировка слиянием очень предсказуема — это её достоинство в реальных системах, где важна стабильная производительность.

Мастер-теорема. Многие алгоритмы вида «разделяй и властвуй» порождают рекуррентность $T(n) = a \cdot T(n/b) + \Theta(n^c)$, где a — число рекурсивных вызовов, b — во сколько раз уменьшается размер, c — степень полиномиальной работы на склейке. Если $c < \log_b a$, ответ — $\Theta(n^{\log_b a})$; если $c = \log_b a$, ответ — $\Theta(n^c \cdot \log n)$; если $c > \log_b a$, ответ — $\Theta(n^c)$. Для сортировки слиянием $a=2$, $b=2$, $c=1$, и мы получаем как раз второй случай: $\Theta(n \log n)$.

Сортировка слиянием расходует дополнительно $O(n)$ памяти на временный массив. Это ограничение можно снять — известны in-place варианты, но они существенно сложнее в реализации.

4.4. Быстрая сортировка

Алгоритм quicksort придумал Чарльз Хоар в 1960 году, ещё студентом, при работе над программой машинного перевода с русского. Идея отличается от merge sort: основная работа делается до рекурсии, а не после.

9. **Выбираем** из массива какой-нибудь элемент — назовём его *опорным* (pivot).
10. **Разбиваем (partition)** массив на две части: слева — элементы, не превосходящие опорный, справа — превосходящие. Опорный встаёт между ними.
11. **Рекурсивно сортируем** обе части. Поскольку элементы из левой части все меньше элементов из правой, отдельная склейка не нужна — массив уже упорядочен.

```
function quicksort(A, lo, hi):
    if lo >= hi: return
    p = partition(A, lo, hi)
    quicksort(A, lo, p - 1)
    quicksort(A, p + 1, hi)

function partition(A, lo, hi):           // схема Ломута
    pivot = A[hi]                        // последний элемент - опорный
    i = lo - 1
    for j = lo to hi - 1:
        if A[j] <= pivot:
            i += 1
            swap(A[i], A[j])
    swap(A[i+1], A[hi])
    return i + 1
```

Пример

Применим quicksort к массиву [3, 7, 1, 8, 5, 2, 6, 4] с опорным элементом 4 (последним). После partition массив выглядит так:



Рис. 4.2. Один шаг быстрой сортировки. После разбиения по опорному элементу 4 слева оказались все меньшие элементы, справа — большие. Дальше каждая часть сортируется тем же методом независимо.

Дальше алгоритм рекурсивно применяется к [3, 1, 2] и к [5, 7, 6, 8]. После их сортировки получится упорядоченный массив [1, 2, 3, 4, 5, 6, 7, 8].

Анализ: лучший, худший, средний случай

Сложность quicksort критически зависит от того, насколько равномерно опорный элемент делит массив.

Лучший случай: опорный — медиана. Тогда массив делится пополам, и мы получаем

$$T(n) = 2 \cdot T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n).$$

Худший случай: опорный — наименьший или наибольший элемент. Например, если массив уже отсортирован и в качестве опорного всегда выбирается последний элемент, partition «отрезает» по одному элементу за раз:

$$T(n) = T(n-1) + \Theta(n) \Rightarrow T(n) = \Theta(n^2).$$

Средний случай (по равномерному распределению входных перестановок) даёт оценку $\Theta(n \log n)$ — её и хочется получить для практики. Аккуратное доказательство этого факта приводится в учебнике Кормена и др. и опирается на следующую идею.

Анализ среднего случая через индикаторы

Пронумеруем элементы массива по их положению после сортировки: $e_1 \leq e_2 \leq \dots \leq e_n$. Введём случайные величины $X_{ij} = 1$, если в процессе работы алгоритма элементы e_i и e_j хотя бы раз сравниваются, и 0 иначе. Общее число сравнений равно $\sum X_{ij}$.

Ключевое наблюдение: e_i и e_j сравниваются ровно тогда, когда один из них первым (среди элементов $e_i, e_{i+1}, \dots, e_j$) выбран опорным. Если же опорным первым становится

какой-нибудь промежуточный элемент e_k ($i < k < j$), то e_i уйдёт в одну подзадачу, а e_j — в другую, и они никогда больше не встретятся.

Если опорный выбирается равновероятно, то среди $j - i + 1$ элементов вероятность того, что первым будет именно e_i или e_j , равна $2/(j - i + 1)$. Значит,

$$E[X_{ij}] = 2 / (j - i + 1).$$

Полное число сравнений в среднем равно

$$E[\sum_{i < j} X_{ij}] = \sum_{i < j} 2/(j - i + 1) = 2 \cdot \sum_i \sum_{k=2..n-i+1} 1/k \approx 2n \cdot (\ln n + \gamma) = \Theta(n \log n).$$

Здесь $\gamma \approx 0.577$ — постоянная Эйлера—Маскерони, и $\sum 1/k$ — гармонический ряд, ведущий себя как $\ln n$. Получили искомый результат: в среднем quicksort работает за $\Theta(n \log n)$ сравнений.

Рандомизированная быстрая сортировка

Чтобы получить хорошее ожидаемое время даже для отсортированных или специально подобранных входов, применяют тот же приём, что и в хеш-таблицах: вносят случайность в сам алгоритм. Перед каждым partition выбирают опорный равномерно случайно из текущего интервала. Тогда независимо от входа ожидаемое время составит $\Theta(n \log n)$ — гарантия по выбору алгоритма, а не по входу.

```
function rand_partition(A, lo, hi):
    r = random_int(lo, hi)
    swap(A[r], A[hi])
    return partition(A, lo, hi)
```

4.5. Сравнение и выводы

Сравним основные характеристики двух алгоритмов сортировки:

Характеристика	Сортировка слиянием	Быстрая сортировка	Quicksort + рандомизация
Лучший случай	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Средний случай	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Худший случай	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^2)$ (но крайне редко)
Доп. память	$\Theta(n)$	$\Theta(\log n)$ (стек)	$\Theta(\log n)$
Стабильна?	да	нет	нет
На практике быстрее	медленнее в константе	обычно быстрее	обычно быстрее

Таблица 4.1. Сравнение сортировок слиянием и быстрой.

Какой алгоритм выбрать на практике? Универсального ответа нет. Стандартные библиотеки чаще всего применяют гибридные алгоритмы, например Timsort (сортировка слиянием с оптимизациями для частично упорядоченных данных, используется в Python и Java) или Introsort (быстрая сортировка с переключением на heapsort при глубокой рекурсии — в C++).

Главный урок параграфа. Принцип «разделяй и властвуй» позволяет сводить задачу размера n к нескольким задачам меньшего размера и через мастер-теорему получать оценки вида $\Theta(n \log n)$ или $\Theta(n^\alpha)$. Анализ среднего случая часто даёт более реалистичную картину, чем «худший случай», особенно если алгоритм рандомизирован.

§5. Задача о рюкзаке: жадность, динамика, приближение

5.1. Постановка задачи

Представьте себе: вы собираетесь в поход и хотите взять рюкзак, выдерживающий не более W килограммов. Перед вами разложено n предметов; для каждого i -го предмета известны его вес w_i и ценность v_i . Можно взять любое подмножество предметов, но суммарный вес не должен превосходить W . Какое подмножество стоит выбрать, чтобы максимизировать суммарную ценность?

Эта задача называется задачей о рюкзаке (knapsack problem) и формализуется так:

$$\max \sum_i v_i \cdot x_i \quad \text{при условии} \quad \sum_i w_i \cdot x_i \leq W, \quad x_i \in \{0, 1\}.$$

Здесь $x_i = 1$ означает, что i -й предмет берётся, $x_i = 0$ — что не берётся. Это вариант 0/1 knapsack: каждый предмет либо целиком берётся, либо не берётся (нельзя взять «половину книги»). Существует и облегчённый вариант — дробный рюкзак, в котором можно брать любую долю предмета. Он, к слову, легко решается жадным алгоритмом, но для нас интересен именно 0/1-вариант: он входит в число тех самых NP-трудных задач, о которых мы скоро будем говорить.

Главное для нас — рюкзак позволит на одном примере увидеть три фундаментальных подхода к решению трудных задач: жадный, динамический и приближённый.

5.2. Жадный алгоритм: почему не работает в общем случае

Самая естественная идея: брать предметы, которые «лучше всего». Что значит «лучше»? Можно сравнивать по ценности, по весу или по их отношению — «удельной ценности» v_i/w_i (сколько ценности приходится на единицу веса). Каноническая жадная стратегия — отсортировать предметы по убыванию удельной ценности и складывать в рюкзак, пока не упрусь в ограничение по весу.

```
function greedy_knapsack(items, W):
    отсортировать items по v/w в убывающем порядке
    chosen = []
    weight = 0
    for item in items:
        if weight + item.w ≤ W:
            chosen.append(item)
            weight += item.w
    return chosen
```

Пример, на котором жадность ошибается

Рассмотрим три предмета и рюкзак $W = 50$:

Предмет	Вес w	Ценность v	v/w	Что делает жадность?
A	10	60	6.0	берём (1-й)
B	20	100	5.0	берём (2-й)
C	30	120	4.0	не помещается

Жадность берёт A и B, общий вес 30, ценность 160. Но лучше выбрать B и C: вес 50, ценность 220! Видно, что простая жадная идея «в лоб» даёт неоптимальный ответ. Можно даже привести примеры, где жадный ответ хуже оптимального в произвольное число раз. Например, при двух предметах с весами 1 и W и ценностями 2 и W жадность возьмёт первый и проиграет.

Значит ли это, что жадные алгоритмы бесполезны? Нет: для дробного рюкзака та же стратегия даёт точный ответ. И часто жадность служит отличной отправной точкой — её результат можно использовать как нижнюю оценку оптимума, а потом улучшать.

5.3. Динамическое программирование

Поскольку «локально лучшее» не всегда даёт «глобально лучшее», нам нужно более гибкое средство. Идея динамического программирования (Ричард Беллман, 1950-е): разложить задачу на подзадачи, ответы для которых хранить в таблице, чтобы не пересчитывать одно и то же дважды.

Введём подзадачу $f(i, w)$: максимальная ценность, которую можно получить, рассматривая только первые i предметов и при ограничении по весу w . Базовый случай: $f(0, w) = 0$ для всех w (нет предметов — нет ценности).

Переход. Пусть мы дошли до i -го предмета. Возможны два сценария:

- **не берём** i -й предмет: тогда ответ совпадает с $f(i - 1, w)$;
- **берём** i -й предмет, если он влезает ($w_i \leq w$): тогда ответ равен $f(i - 1, w - w_i) + v_i$.

Лучшее из двух — наш ответ:

$f(i, w) = \max\{ f(i-1, w), f(i-1, w - w_i) + v_i \} \quad (\text{если } w_i \leq w),$
$f(i, w) = f(i - 1, w), \quad \text{если } w_i > w.$

Ответ задачи — это $f(n, W)$. Стоимость вычисления — $n \cdot (W + 1)$ клеток таблицы, каждая считается за $O(1)$. Итого сложность $\Theta(n \cdot W)$ операций и $\Theta(n \cdot W)$ памяти.

Пошаговый пример

Возьмём 4 предмета с $(w, v) = (2, 3), (3, 4), (4, 5), (5, 6)$ и рюкзак $W = 5$. Заполним таблицу $f[i][w]$:

Заполнение таблицы DP[i][W] для задачи о рюкзакеПредметы: $w=[2,3,4,5]$, $v=[3,4,5,6]$, $W=5$

i\W	0	1	2	3	4	5
—	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

Ответ = 7

Рис. 5.1. Заполнение DP-таблицы для задачи о рюкзаке. Зелёным выделена итоговая ячейка — оптимальное значение.

Поясним заполнение для нескольких ключевых клеток. $f[1][2] = \max\{f[0][2], f[0][0] + 3\} = \max\{0, 3\} = 3$ (взяли первый предмет, веса 2). $f[2][5] = \max\{f[1][5], f[1][2] + 4\} = \max\{3, 7\} = 7$ (взяли первый и второй предметы).

Восстановление состава решения. Идём по таблице от ячейки $f[n][W]$ обратно: если $f[i][w] \neq f[i-1][w]$, значит, i -й предмет был взят, и переходим в $f[i-1][w - w_i]$; иначе — переходим в $f[i-1][w]$. Так за $O(n)$ шагов получим список выбранных предметов. В нашем примере оптимум 7 даёт набор {1-й, 2-й} с весами $2+3=5$.

Сложность и важная оговорка

Алгоритм работает за $O(n \cdot W)$ операций. Это полиномиально по n и W . Но обратите внимание: W — это число, и число это может быть очень большим. Если $W = 2^{100}$ (длина записи — 100 бит), то « $O(n \cdot W)$ » операций недостижимо за человеческое время. Сложность зависит от значения числа W , а не от длины его записи. Такие алгоритмы называют псевдополиномиальными: они полиномиальны по числовому значению входа, но экспоненциальны по битовой длине.

Принцип динамического программирования. Если оптимальное решение задачи можно построить из оптимальных решений подзадач — пусть и не очевидным образом, — то имеет смысл сосчитать ответы для всех подзадач и записать в таблицу. Это меняет экспоненциальное число вариантов перебора на полиномиальное число клеток. Таблица — это «память» алгоритма, которая запоминает уже сделанную работу.

5.4. ϵ -приближённый алгоритм

Псевдополиномиальный алгоритм работает быстро только при сравнительно небольшом W . А что делать, если W велико, но мы готовы согласиться на чуть-чуть неоптимальный ответ?

Здесь рождается идея приближённых алгоритмов. Алгоритм называется ϵ -приближённым ($\epsilon > 0$) для задачи на максимум, если он гарантирует, что выдаваемое значение ALG не меньше $(1 - \epsilon) \cdot OPT$, где OPT — точный оптимум. Чем меньше ϵ , тем ближе к оптимуму ответ.

Если для любого $\epsilon > 0$ существует алгоритм, работающий за время, полиномиальное от n и от $1/\epsilon$, говорят, что задача имеет полностью полиномиальную приближённую схему (FPTAS — fully polynomial time approximation scheme). Удивительно, но для задачи о рюкзаке такая схема существует.

Идея масштабирования ценностей

Из формулы DP видно, что узкое место — большое W . А что, если бы вместо W узким местом было V — наибольшая ценность предмета? Можно построить «двойственную» DP по ценностям:

Введём $g(i, v)$ — наименьший суммарный вес, при котором, рассматривая первые i предметов, можно набрать ровно ценность v . Тогда ответ — это $\max\{v : g(n, v) \leq W\}$. Таблица имеет размер $n \cdot n \cdot v_{\max} \leq n^2 \cdot V$, где $V = \max v_i$. Стоимость одного обновления — $O(1)$. Итого $O(n^2 \cdot V)$ — полиномиально по n и V .

Эта идея не помогает, если V велико. Но раз она зависит от значений ценностей, можно эти значения уменьшить — округлить — и получить чуть менее точный, но быстрый алгоритм.

Алгоритм

```
function fptas_knapsack(items, W, ε):
    V = max v_i
    K = ε · V / n           // масштаб округления
    для каждого предмета: v'_i = floor(v_i / K)
    решить задачу о рюкзаке (n, W, w, v') методом DP по ценностям
    в качестве ответа выдать выбранный набор предметов
```

Что мы тут сделали? Мы заменили истинные ценности v_i на «огрубленные» $v'_i = \lfloor v_i / K \rfloor$. Это уменьшает максимум ценности до $V/K = n/\epsilon$, и DP по ценностям работает за $O(n^2 \cdot n/\epsilon) = O(n^3/\epsilon)$. Это полиномиально и от n , и от $1/\epsilon$.

Анализ точности

Покажем, что ответ не хуже $(1 - \epsilon) \cdot OPT$. Пусть S^* — оптимальный набор, $OPT = \sum_{i \in S^*} v_i$. Округление искажает каждую ценность не более чем на K , поэтому

$$K \cdot v'_i \leq v_i \leq K \cdot v'_i + K.$$

Алгоритм находит точный максимум огрублённой задачи; обозначим найденный набор $\hat{S}$. Тогда суммарная огрублённая ценность $\hat{S}$ не меньше суммарной огрублённой ценности S^* :

$$\sum_{i \in \hat{S}} v'_i \geq \sum_{i \in S^*} v'_i.$$

Умножим обе части на K и применим неравенства из определения округления:

$$\sum_{i \in \hat{S}} v_i \geq K \cdot \sum_{i \in \hat{S}} v'_i \geq K \cdot \sum_{i \in S^*} v'_i \geq \sum_{i \in S^*} (v_i - K) = \text{OPT} - n \cdot K.$$

Подставим $K = \varepsilon \cdot V / n$. Получаем

$$\text{ALG} \geq \text{OPT} - n \cdot \varepsilon V/n = \text{OPT} - \varepsilon \cdot V.$$

Поскольку $V \leq \text{OPT}$ (потому что один предмет максимальной ценности всегда можно положить в рюкзак, если он туда влезает; в противном случае задача тривиальна), получаем $\text{ALG} \geq (1 - \varepsilon) \cdot \text{OPT}$, что и требовалось доказать.

Пример

Пусть $n = 3$ предмета, $W = 100$, $\varepsilon = 0.1$. Ценности: $v = (24, 50, 99)$. Значит $V = 99$, $K = 0.1 \cdot 99/3 = 3.3$. Округлённые ценности $v' = \lfloor 24/3.3 \rfloor, \lfloor 50/3.3 \rfloor, \lfloor 99/3.3 \rfloor = 7, 15, 30$. Вместо чисел до 99 теперь работаем с числами до 30. Это не сильно ускоряет на таком маленьком примере, но при V порядка 10^6 выигрыш огромный: вместо 10^6 значений — лишь $n/\varepsilon = 30$.

Большая идея приближения. Если точно решить задачу за полиномиальное время не получается, но мы готовы уступить в точности — иногда можно построить алгоритм с произвольной заданной точностью ε и при этом полиномиальный по $1/\varepsilon$. Это очень мощная парадигма; для многих NP-трудных задач (вершинное покрытие, упаковка корзины, разбиение) известны подобные схемы.

5.5. Сводная таблица

Алгоритм	Сложность	Точность	Достоинства / недостатки
Жадный по v/w	$O(n \log n)$	в общем случае не гарантирует ничего	прост, быстр, но может ошибиться сколь угодно сильно
DP по весам	$O(n \cdot W)$	точный	работает только при разумном W
DP по ценностям	$O(n^2 \cdot V)$	точный	работает только при разумном V
FPTAS (масштабирование)	$O(n^3/\varepsilon)$	$(1-\varepsilon) \cdot \text{OPT}$	любая желаемая точность за полиномиальное время

Таблица 5.1. Алгоритмы для задачи о рюкзаке.

§6. P, NP и за их пределами. Доказательства без разглашения

6.1. Класс P: «эффективно решаемые» задачи

Мы уже встречались с границей между полиномиальным и экспоненциальным временем работы. Чтобы говорить о ней строго, нужно уточнить, какие задачи мы вообще обсуждаем. Удобнее всего ограничиться задачами распознавания: на вход подаётся слово (последовательность символов), на выход требуется ответ «да» или «нет».

Класс P состоит из тех задач распознавания, для которых существует алгоритм, дающий правильный ответ за время, ограниченное некоторым многочленом от длины входа. Например, «является ли заданное число чётным?» — задача из P. «Связан ли заданный граф?» — тоже из P (поиск в ширину работает за $O(V + E)$). «Проверка простоты числа?» — этот пункт мы обсудим отдельно.

Алгоритмы из всех предыдущих параграфов — алгоритм Евклида, поиск в хеш-таблице, сортировки слиянием и быстрая (с подходящей реализацией), даже точное динамическое программирование для рюкзака с малым W — формально работают за полиномиальное время. Но «полиномиальное по чему?» — это важный вопрос. Полиномиальная сложность определяется относительно длины входа в битах. Если W записано $L = \lceil \log_2 W \rceil$ битами, то $O(n \cdot W) = O(n \cdot 2^L)$, и это уже не полиномиально. Поэтому мы и говорили выше про псевдополиномиальность.

6.2. Класс NP: «легко проверяемые» задачи

Многие интересные задачи кажутся вычислительно тяжёлыми, но обладают одним общим свойством: ответ «да» легко проверить, если вам подсказали правильное доказательство. Например, для задачи «есть ли в графе клика размера k » (то есть подграф из k попарно соединённых вершин) сама проверка тривиальна: дайте мне эти k вершин, и я за $O(k^2)$ операций проверю наличие всех нужных рёбер. Найти такую клику — совсем другая задача, заведомо более трудная.

Класс NP — это совокупность всех задач, для которых ответ «да» можно подтвердить с помощью некоторого «сертификата» (подсказки) за полиномиальное время. Аббревиатура NP расшифровывается как *nondeterministic polynomial* («полиномиально на недетерминированной машине»): можно представить себе фантастическую машину, которая «угадывает» правильный сертификат и затем проверяет его за полиномиальное время.

Очевидно, что $P \subseteq NP$: если задачу можно быстро решить, то полученный ответ можно использовать как сертификат. Обратное включение — главный нерешённый

вопрос: верно ли, что $NP \subseteq P$? Это и есть знаменитая проблема P vs NP . Большинство специалистов считает, что $P \neq NP$, то есть существуют задачи, ответ для которых легко проверить, но трудно найти. Однако строгого доказательства до сих пор нет.

6.3. NP-полные задачи и сводимость

Среди всех задач из NP есть особый подкласс «самых трудных» — NP -полных. Чтобы определить их, нам нужно понятие сводимости.

Говорят, что задача A сводится за полиномиальное время к задаче B ($A \leq_p B$), если существует эффективное преобразование экземпляров задачи A в экземпляры задачи B , сохраняющее ответ. Иначе говоря, любой алгоритм для B можно использовать для решения A : достаточно перед вызовом «отконвертировать» вход.

Задача L называется NP -полной, если:

12. $L \in NP$;

13. любая задача $L' \in NP$ сводится за полиномиальное время к L .

Содержательно: если бы мы умели решать какую-то одну NP -полную задачу за полиномиальное время, мы умели бы все задачи из NP . То есть NP -полные задачи — это «самые твёрдые орешки».

В 1971 году Стивен Кук (а независимо Леонид Левин в 1973-м) доказал, что задача SAT — выполнимость булевой формулы в конъюнктивной нормальной форме — NP -полна. Это первое в истории NP -полное утверждение. Уже на следующий год Карп показал NP -полноту 21 классической задачи: разбиения числа, гамильтонова цикла, задачи коммивояжёра, раскраски графа, клики, вершинного покрытия, целочисленного линейного программирования...

К нашему списку: задача о рюкзаке тоже NP -полна. Поэтому жадные методы для неё не дают точного ответа в общем случае, и не существует (если $P \neq NP$) полиномиального алгоритма, всегда дающего точный оптимум. Зато, как мы видели, существуют псевдополиномиальные и приближённые алгоритмы.

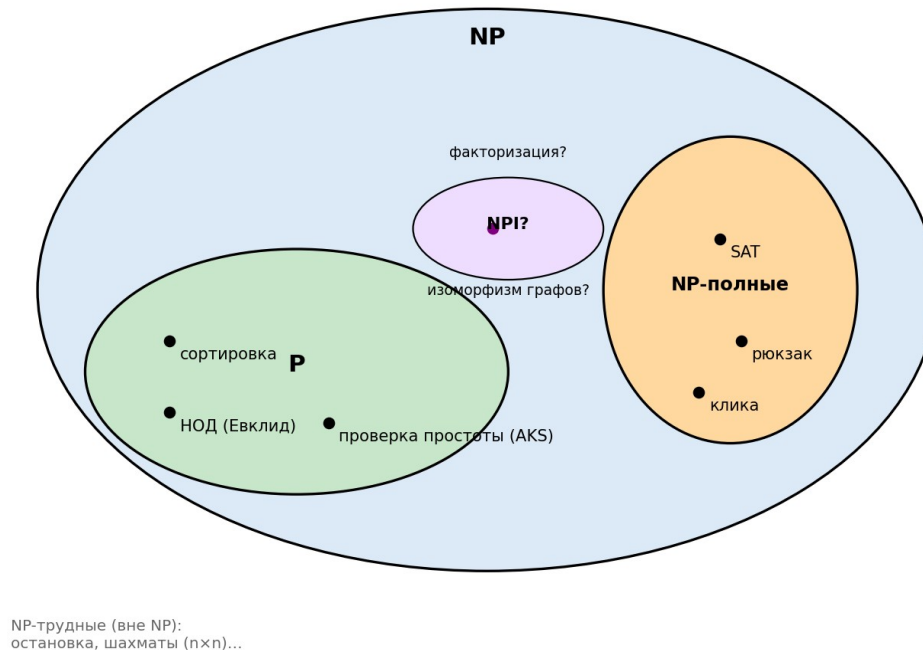
Гипотетическая структура классов (если $P \neq NP$)

Рис. 6.1. Гипотетическое устройство классов сложности при предположении $P \neq NP$. Внутри NP лежит класс P (зелёный); справа — NP-полные задачи (оранжевый); посередине, возможно, есть «промежуточный» класс NP? (фиолетовый), к которому подозревают принадлежность факторизации и изоморфизма графов.

6.4. Зачем знать, что задача NP-трудна

Знание того, что ваша задача NP-трудна, — это не приговор, а информация. Она говорит: не тратьте силы на поиск общего полиномиального алгоритма, его, скорее всего, не существует. Но возможны другие подходы:

- Эвристики и алгоритмы локального поиска: они часто на практике дают хорошие результаты, не имея теоретической гарантии.
- Точные алгоритмы экспоненциальной сложности с малой постоянной (например, известны алгоритмы для SAT, работающие за $O(1.3^n)$).
- Приближённые алгоритмы с гарантированной точностью — мы видели пример с рюкзаком.
- Параметризованные алгоритмы: задача может быть полиномиально решаемой при фиксированном значении некоторого параметра.
- Использование структуры конкретного входа: «общий» вход труден, но реальные входы часто обладают свойствами, упрощающими решение.

6.5. Задача факторизации и проверка простоты

Эти две задачи о натуральных числах часто упоминают в одной строке, но они принципиально различаются по сложности.

Проверка простоты

Дано число N в десятичной (или двоичной) записи. Является ли оно простым? До 2002 года был известен только полиномиальный вероятностный алгоритм Миллера—Рабина, который для каждой проверки даёт ответ «возможно простое» с очень высокой вероятностью. В 2002 году Маниндра Агравал, Нирадж Каял и Нитин Саксена опубликовали свой алгоритм AKS — детерминированный полиномиальный тест простоты. Так задача проверки простоты официально вошла в класс P .

Это была сенсация. Ещё в школьных условиях мы можем приближаться к простоте через быстрые вероятностные тесты, но именно AKS показал, что такая возможность существует и теоретически. Алгоритм AKS работает за полиномиальное время от длины записи N (то есть от $\log N$), хотя в практических вычислениях он медленнее, чем вероятностные тесты — поэтому для приложений по-прежнему пользуются Миллер—Рабином.

Факторизация составного числа

Совсем другая история — задача найти разложение числа N на простые множители. Сейчас неизвестен полиномиальный (от длины записи N) детерминированный алгоритм для этого. Лучшие классические алгоритмы (общее решето поля чисел) работают за время $\exp(O((\log N)^{1/3} (\log \log N)^{2/3}))$ — так называемая «субэкспоненциальная» сложность. Это далеко от полиномиальной.

Именно на этом несоответствии — «легко проверить, что (p, q) — разложение N (просто перемножим), но трудно его найти» — построена криптосистема RSA. Если бы факторизация была в P , мы могли бы взламывать RSA за полиномиальное время. С другой стороны, факторизация лежит в NP , потому что разложение служит самим сертификатом. Известно ли, что факторизация — NP -полная? Нет, и это дополнительный сюрприз: задача факторизации, по-видимому, занимает «промежуточное» положение между P и NP -полными — её относят к гипотетическому классу NPI («промежуточные», NP -intermediate).

Алгоритм Шора (1994). Питер Шор показал, что на квантовом компьютере факторизация решается за полиномиальное время. Этот результат — главный аргумент за «постквантовую криптографию», и одна из причин, по которым работают огромные программы по созданию квантовых вычислителей.

6.6. Задача изоморфизма графов

Два графа G_1 и G_2 называются изоморфными, если можно так перенумеровать вершины одного, чтобы получить второй (то есть существует биекция ϕ между множествами вершин, сохраняющая рёбра). На вид это «всего лишь» проверка одинаковости графов с точностью до названий вершин — но задача оказалась удивительно тонкой.

Изоморфные графы: можно ли быстро это распознать?

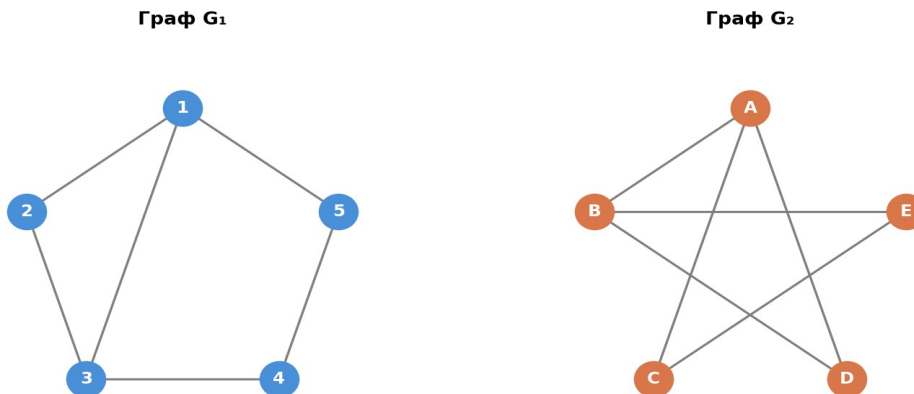


Рис. 6.2. Изоморфизм графов. Графы G_1 и G_2 изоморфны: соответствие $1 \leftrightarrow B, 2 \leftrightarrow D, 3 \leftrightarrow A, 4 \leftrightarrow C, 5 \leftrightarrow E$ сохраняет все рёбра. Но как для произвольных больших графов быстро проверить, есть ли такое соответствие?

Изоморфизм графов лежит в NP (в качестве сертификата выступает биекция ϕ — её проверка занимает $O((V + E))$). Является ли он NP-полным или, наоборот, лежит в P — десятилетиями оставалось открытым. В 2015 году Ласло Бабай объявил алгоритм с квазиполиномиальной сложностью $\exp((\log n)^{O(1)})$, что меньше любой степени, но больше многочлена. Это великий результат — но всё-таки не полиномиальный алгоритм. Большинство теоретиков склоняются к тому, что задача находится в загадочном классе NPI: не в P и не NP-полная.

6.7. Доказательства с нулевым разглашением

Что бы вы стали делать, если бы захотели убедить кого-то, что владеете секретом, не выдавая самого секрета? Звучит как парадокс, и ровно это противоречие разрешается криптографическим протоколом, известным как доказательство с нулевым разглашением (zero-knowledge proof). Понятие ввели Шафи Голдвассер, Сильвио Микали и Чарли Раккоф в 1985 году.

Пример: пещера Али-Бабы

Этот сюжет (его в 1990 году придумали Жан-Жак Кискатер и его соавторы) хорошо иллюстрирует идею. Представьте себе кольцеобразную пещеру: единственный

вход разветвляется на два коридора А и В, которые в дальнем конце соединены волшебной дверью. Дверь открывается лишь по секретному заклинанию.

«Пещера Али-Бабы»: схема нулевого разглашения

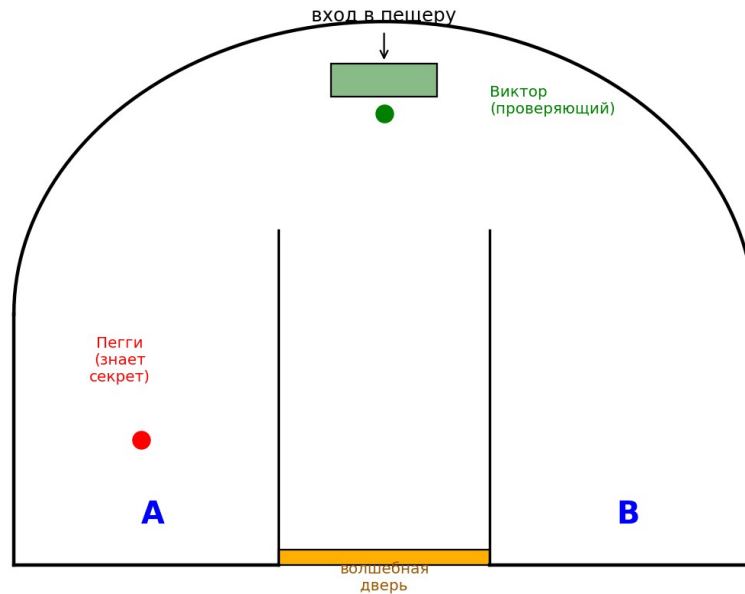


Рис. 6.3. «Пещера Али-Бабы». Пегги (от англ. Prover) утверждает, что знает заклинание; Виктор (Verifier) хочет в этом убедиться, не услышав самого заклинания.

Пегги (доказывающая) хочет убедить Виктора (проверяющего) в том, что она знает заклинание. Сделать это, просто сказав слово, нельзя: тогда Виктор узнает секрет. Применим следующий протокол:

14. Виктор стоит у входа и не видит, в какой коридор ушла Пегги. Пегги случайно выбирает А или В и углубляется в один из них.
15. Виктор подходит к развилке и кричит: «Возвращайся через А» или «Возвращайся через В» — на выбор.
16. Если Пегги действительно знает заклинание, она всегда может выйти через нужный коридор: если она в нужном — просто идёт обратно, если в другом — открывает дверь и переходит.
17. Если Пегги обманывает (заклинания не знает), она угадает требуемый коридор лишь с вероятностью $1/2$.

Что мы получили? Если Пегги обманывает, риск разоблачения — 50% за один раунд. Повторим протокол n раз. Тогда риск обмана снизится до 2^{-n} — пренебрежимо малая величина. И при этом Виктор так и не узнал заклинания! Информация о секрете полностью равна нулю — отсюда и название «zero-knowledge».

Свойства доказательств без разглашения

Формально доказательство с нулевым разглашением должно удовлетворять трём свойствам:

- **Полнота (completeness):** если утверждение истинно и Пегги ведёт себя честно, Виктор всегда соглашается.
- **Корректность (soundness):** если утверждение ложно, никакая стратегия Пегги не убедит Виктора с вероятностью больше очень малой (например, 2^{-n}).
- **Нулевое разглашение (zero-knowledge):** Виктор после взаимодействия не получает никакой полезной информации, кроме факта истинности утверждения. Формально: всё, что мог бы узнать Виктор, можно «симулировать» без участия Пегги.

ZK-доказательство для изоморфизма графов

Покажем настоящий криптографический протокол на основе задачи об изоморфизме. Допустим, Пегги знает изоморфизм ϕ между двумя графами G_1 и G_2 (то есть знает биекцию $\phi: V_1 \rightarrow V_2$, переводящую рёбра в рёбра) и хочет убедить в этом Виктора, не выдавая саму ϕ . Один раунд протокола устроен так:

18. Пегги случайно генерирует биекцию ψ и применяет её к G_1 : получает граф $H = \psi(G_1)$. Заметим: H одновременно изоморфен и G_1 (через ψ), и G_2 (через $\psi \circ \phi^{-1}$).
19. Пегги отправляет H Виктору, не раскрывая, как он построен.
20. Виктор бросает монету и просит Пегги показать либо изоморфизм $H \leftrightarrow G_1$, либо изоморфизм $H \leftrightarrow G_2$.
21. Пегги отвечает: для первого запроса присылает ψ , для второго — композицию $\psi \circ \phi^{-1}$.
22. Виктор проверяет полученный изоморфизм: если он действительно изоморфизм — раунд считается пройденным.

Если G_1 и G_2 не изоморфны, Пегги не может одновременно подготовить и $H \leftrightarrow G_1$, и $H \leftrightarrow G_2$; шансы её обмана — $1/2$ за раунд. После n раундов вероятность обмана 2^{-n} . При этом Виктор узнаёт лишь по одному изоморфизму к случайному H за раунд — это случайные графы и случайные биекции, никакой информации о настоящем ϕ .

Где это применяется

Доказательства с нулевым разглашением — это не школьная диковинка, а основа важных современных систем.

- Криптовалюты с приватностью (Zcash и подобные) используют zk-SNARK — компактные доказательства без разглашения, чтобы клиенты могли подтверждать корректность транзакций, не выдавая ни сумм, ни адресов.
- Системы аутентификации без передачи пароля (например, в стандарте FIDO2).

- Электронное голосование с обеспечением и тайны, и проверяемости.
- Подтверждение возраста или гражданства без передачи документа.

Важная мысль. На «трудности» некоторых задач (факторизация, дискретный логарифм, изоморфизм графов) построены целые отрасли человеческой деятельности. Если, скажем, окажется, что $P = NP$, рухнет почти вся современная криптография: всякий публичный ключ можно будет взломать. С другой стороны, доказательство $P \neq NP$ стало бы математическим откровением. Поэтому теория сложности — не абстрактные занятия математиков, а опора нашего цифрового мира.

Заключение. Большие идеи теории сложности

Глава, которую вы прочитали, охватила далеко не всё богатство современной теории алгоритмов. Но если попробовать выделить главное — то получится небольшой набор сквозных идей, без которых невозможно представить себе ни один курс «продвинутого» уровня.

Идея 1. Сложность измеряется по «правильной» оси.

Алгоритм невозможно оценивать по числу секунд на конкретном компьютере: компьютеры разные, а задачи — большие. Поэтому фундаментальная мера — асимптотический рост числа операций как функции длины входа. Этот сдвиг точки зрения, родом из работ Хартманиса и Стирнза, сделал теорию алгоритмов наукой, а не сборником рецептов.

Идея 2. Разделяй и властвуй.

Если задачу размера n удаётся свести к нескольким подзадачам существенно меньшего размера и недорого «склеить» их решения — есть шанс получить алгоритм со сложностью $O(n \log n)$ или даже лучше. Сортировка слиянием, быстрое умножение, быстрое преобразование Фурье, поиск замощений и многое другое — всё это потомки одной идеи.

Идея 3. Динамическое программирование.

Когда задачу нельзя элегантно разделить, но можно описать её решение через «оптимальные ответы для меньших экземпляров» — стоит попробовать построить таблицу. Так появились решения задач о рюкзаке, выравнивания строк, кратчайших путей, оптимального произведения матриц. Динамическое программирование часто превращает экспоненциальный перебор в полиномиальный пересчёт.

Идея 4. Рандомизация.

Случайность — мощный инструмент. Случайный выбор опорного элемента в быстрой сортировке делает её математически устойчивой к плохим входам. Случайный выбор хеш-функции из универсального семейства гарантирует малое число коллизий вне зависимости от данных. Случайные тесты простоты дают эффективные алгоритмы там, где детерминированные ещё недавно казались недостижимыми. Доказательства без разглашения тоже опираются на случайность — без неё они были бы попросту невозможны.

Идея 5. Анализ в среднем и амортизация.

Худший случай — важный, но не единственный ориентир. Иногда среднее по входам или по случайным выборам алгоритма гораздо честнее отражает практическое

поведение. Этот тип анализа сделал быструю сортировку, хеш-таблицы и многие структуры данных по-настоящему «массовыми» инструментами.

Идея 6. Полиномиальная редукция.

Если мы умеем эффективно превращать задачу А в задачу В, то умение решать В даёт нам решение А. Так удалось связать тысячи разных по виду задач в единое NP-полное «семейство» и понять: либо все они быстро решаются ($P = NP$), либо ни одна ($P \neq NP$). Сводимости — основной язык современной теории сложности.

Идея 7. Приближение там, где точное решение недостижимо.

Если NP-трудную задачу нельзя решить точно за полиномиальное время, можно попытаться решить её приближённо. FPTAS для рюкзака — пример такого подхода: чем точнее ответ нам нужен, тем больше времени мы готовы потратить, но всё равно полиномиально по размеру входа. Аналогично устроены приближённые алгоритмы для коммивояжёра, разрезов, покрытий и многого другого.

Идея 8. Доказательство без разглашения.

Совсем недавно, в исторических масштабах, оказалось, что можно доказывать утверждения, не сообщая никаких сведений сверх их истинности. Это идея переворачивает наше представление о понятии «доказательства» и уже стала технологической основой блокчейн-систем, протоколов аутентификации и электронного голосования.

За пределами этой главы остались целые ветви — параллельные алгоритмы, потоковые алгоритмы, алгоритмы для задач больших данных, квантовые алгоритмы, online-алгоритмы, алгоритмы машинного обучения. Мы надеемся, что освоенные принципы помогут вам при их самостоятельном изучении.

Главное, что мы хотели показать. Алгоритм — это не способ «заставить компьютер посчитать»: это способ доказать математическое утверждение о существовании эффективного решения. Анализ сложности — это разговор о реальных границах возможного. И хотя вычислительные мощности растут год от года, ровно те же законы — экспоненциальный рост остаётся экспоненциальным, а полиномиальный — полиномиальным — определяют, какие задачи компьютеры будут уметь решать через сто лет, а какие — никогда.

Литература

Если изложенный материал заинтересовал вас, то для дальнейшего изучения мы рекомендуем следующие источники.

Основная литература

1. **Дасгупта С., Пападимитриу Х., Вазирани У.** Алгоритмы. — Москва: МЦНМО, 2014.

Превосходный современный учебник, идеально подходящий для первого знакомства с теорией алгоритмов. Все алгоритмы изложены кратко, прозрачно, с акцентом на идеи, а не на технические детали. Особенно хороши главы о делении и властуй, динамическом программировании, NP-полных задачах и квантовых алгоритмах.

2. **Кузюрин Н. Н., Фомин С. А.** Эффективные алгоритмы и сложность вычислений. — Москва: МФТИ, 2017.

Систематический курс по сложности вычислений на русском языке. Включает строгое изложение классов P, NP, NP-полноты, рандомизированных алгоритмов, приближённых алгоритмов, а также главы по криптографии и интерактивным доказательствам. Идеален как «второе чтение» после Дасгупты.

3. **Математические этюды.** Сложность алгоритмов. — Электронное издание. — URL: <https://book.etudes.ru/articles/complexity/>

Прекрасные интерактивные иллюстрации к алгоритму Евклида, сортировкам, P vs NP и другим темам. Очень рекомендуется как дополнение к чтению: многие идеи становятся куда более ясными, когда видишь их «в движении».

Дополнительная литература

4. **Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К.** Алгоритмы: построение и анализ. 3-е изд. — Москва: Вильямс, 2013.

Знаменитая «толстая книга» по алгоритмам. Энциклопедически полна и точна; при этом не предназначена для последовательного чтения от корки до корки. Используйте как справочник.

5. **Кнут Д.** Искусство программирования. Том 1: Основные алгоритмы. — Вильямс, 2018.

Классика, которую полезно держать на полке. Многие задачи, разобранные у Кнута (включая алгоритм Евклида и сортировки), позволяют увидеть глубокую математическую культуру за самыми простыми, казалось бы, программами.

6. **Goldreich O.** Computational Complexity: A Conceptual Perspective. — Cambridge University Press, 2008.

Современный концептуальный учебник по теории сложности. Очень рекомендуется тем, кто хочет глубже погрузиться в P vs NP , интерактивные доказательства и доказательства без разглашения.

7. **Babai L.** Graph Isomorphism in Quasipolynomial Time. — arXiv:1512.03547, 2015.

Оригинальная статья Ласло Бабаи, в которой получен квазиполиномиальный алгоритм для задачи изоморфизма графов. Чтение требует специальной подготовки, но первые разделы доступны и старшеклассникам.

Задачи для самостоятельной работы

Решение задач — лучший способ закрепить материал. Задачи разделены на блоки по параграфам; уровень сложности обозначен значками: ° — на проверку понимания, * — задача среднего уровня, ** — задача повышенной трудности.

К §1 (основные понятия)

1. ° Сравните функции n^2 , $n \log^2 n$, $1000 \cdot n$, $2^{\sqrt{n}}$. Какая из них растёт быстрее остальных при достаточно больших n ? Расположите функции в порядке роста.
2. ° Алгоритм А работает за $O(n^2)$, алгоритм В — за $O(n^3)$. При каких размерах входа алгоритм В всё ещё может оказаться быстрее А на практике? Объясните, чем не противоречит этому асимптотическая оценка.
3. * Покажите, что если $f(n) = O(g(n))$ и $g(n) = O(h(n))$, то $f(n) = O(h(n))$. Тем самым отношение «расти не быстрее» транзитивно.
4. * Докажите, что любой алгоритм поиска элемента в неотсортированном массиве в худшем случае выполняет не менее $n - 1$ сравнений с искомым.

К §2 (алгоритм Евклида)

5. ° Найдите НОД(143, 91) и НОД(1071, 462) с помощью алгоритма Евклида. Запишите все шаги.
6. ° Решите расширенным алгоритмом Евклида диофантово уравнение $17x + 5y = 1$.
7. * Найдите обратный элемент к 7 по модулю 26. Используйте расширенный алгоритм Евклида.
8. * Докажите, что для любых натуральных $a > b$ число шагов в алгоритме Евклида не превосходит $5 \cdot \log_{10} b$. (Это уточнённая оценка теоремы Ламе.)
9. ** Сколько в среднем шагов делает алгоритм Евклида при равномерно случайном выборе пары (a, b) с $a, b \leq N$? (Ответ: имеет порядок $\log N$ с константой Портера $\approx 0,5854\dots$)

К §3 (универсальное хеширование)

10. ° Реализуйте хеш-таблицу с цепочками для целых ключей с хеш-функцией $h(x) = x \bmod m$. Для $m = 10$ и ключей 23, 51, 38, 47, 12, 95, 31, 74 покажите итоговую структуру таблицы.
11. * Найдите такую последовательность из 8 ключей, на которой обычная функция $h(x) = x \bmod 10$ даёт максимально длинную цепочку. Что это говорит о применимости детерминированного хеширования?
12. * Покажите, что семейство $h_{\{a,b\}}(x) = ((a \cdot x + b) \bmod p) \bmod m$ из §3 действительно универсально: вероятность коллизии не превосходит $1/m$.

13. ** Постройте универсальное семейство хеш-функций для строк длины $\leq \ell$. (Подсказка: считайте строку числом в системе счисления с большим основанием.)

К §4 (сортировки)

14. ° Отсортируйте массив [5, 2, 9, 1, 7, 3, 8, 4, 6] алгоритмом сортировки слиянием. Нарисуйте дерево рекурсии.
15. ° Отсортируйте тот же массив быстрой сортировкой, выбирая в качестве опоры последний элемент.
16. * Покажите, что для сортировки слиянием $T(n) = 2T(n/2) + \Theta(n)$ даёт $T(n) = \Theta(n \log n)$. (Используйте мастер-теорему или рекурсивное дерево.)
17. * Найдите такой вход размера n , на котором быстрая сортировка с выбором опоры «всегда первый элемент» работает за $\Theta(n^2)$ сравнений.
18. ** Докажите, что любой алгоритм сортировки сравнениями в худшем случае выполняет $\Omega(n \log n)$ сравнений. (Подсказка: дерево решений с $n!$ листьями.)

К §5 (рюкзак)

19. ° Решите задачу о рюкзаке вместимостью $W = 7$ для предметов с весами и стоимостями (2, 3), (3, 4), (4, 5), (5, 6) тремя способами: жадно по v/w , динамическим программированием и полным перебором. Сравните результаты.
20. * Постройте такой пример из 5 предметов, на котором жадный алгоритм по отношению v/w ошибается более чем вдвое.
21. * Оцените, сколько ячеек DP-таблицы потребуется для рюкзака с $W = 10^9$ и $n = 50$. Реалистично ли это? Какой алгоритм здесь предпочтительнее?
22. ** Реализуйте FPTAS для рюкзака с параметром $\epsilon = 0.1$ и проверьте на нескольких случайных входах, что приближение действительно укладывается в $(1 - \epsilon) \cdot \text{ОПТ}$.

К §6 (P vs NP, криптография)

23. ° Какие из следующих задач принадлежат P, а какие — NP? Связан ли граф; есть ли в графе клика размера k ; найти кратчайший путь; найти максимальный гамильтонов цикл; проверить простоту числа.
24. * Покажите, что задача «есть ли в графе подграф из k попарно не соединённых вершин» (независимое множество) NP-полна, сводя к ней задачу о клике.
25. * Покажите, что если $P = NP$, то существует полиномиальный алгоритм, не только проверяющий, но и находящий разложение составного числа на простые множители.

26. * В протоколе Али-Бабы Виктор тридцать раз подряд получает правильный ответ. Какова вероятность того, что Пегги его обманывает (не знает пароля)?
27. ** Опишите протокол доказательства без разглашения для задачи «трёхраскраски графа». В чём состоит обязательство и каков «вызов» проверяющего? Чему равна вероятность обмана за один раунд?

Удачи в изучении алгоритмов! Помните: настоящая глубина теории сложности раскрывается тем, кто не боится «сесть и подумать» над задачей хотя бы час, прежде чем заглянуть в подсказку.